

Расчет ресурсов и оптимизация инференса LLM

AI Архитектор



Проверить, идет ли запись

Меня хорошо видно & слышно?





Иван Четвериков

AI Architect

- Более 5 лет опыта в разработке и архитектуре
- Специалист в разработке и проектировании AI-приложений в Raft

 @istrebitel_1

 istrebitel.3.12@gmail.com



Правила вебинара



Активно
участвуем



Off-topic обсуждаем
в учебной группе



Задаем вопрос
в чат или голосом



Вопросы вижу в чате,
могу ответить не сразу

Маршрут вебинара

Знакомство

Расчет VRAM

Квантование

Техники оптимизации инференса



Цели вебинара

К концу занятия вы сможете

1. Рассчитывать сайзинг необходимый
для запуска LLM

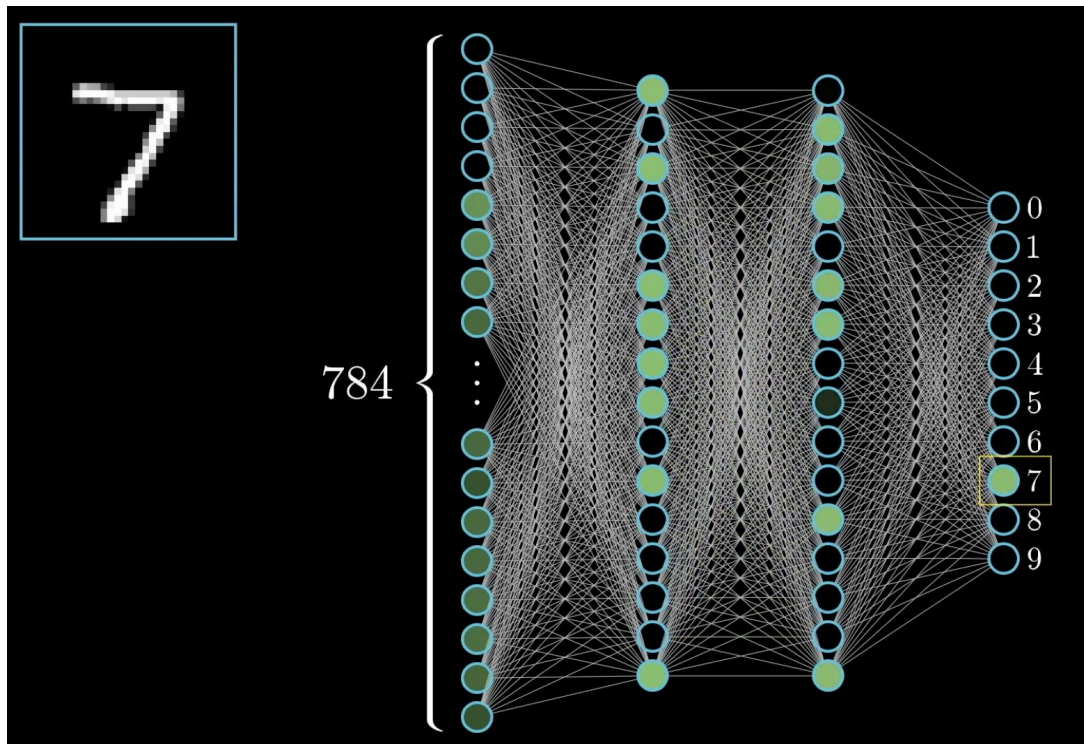
2. Оптимизировать обучение LLM
моделей

3. Оптимизировать инференс LLM
моделей



Расчет сайзинга VRAM

Веса модели



Веса модели, на примере GPT-3

- Матрица эмбеддера – 617M
- Позиционные эмбеддинги – 25M
- Головы внимания (Q, K, V) – 605M
- Нейроны нелинейных преобразований (FFN) – 1208M
- Вектора нормализации – 49K

Итого – $617M + 25M + 96 \cdot (605M + 1208M + 49K) = 175B$



Формула VRAM

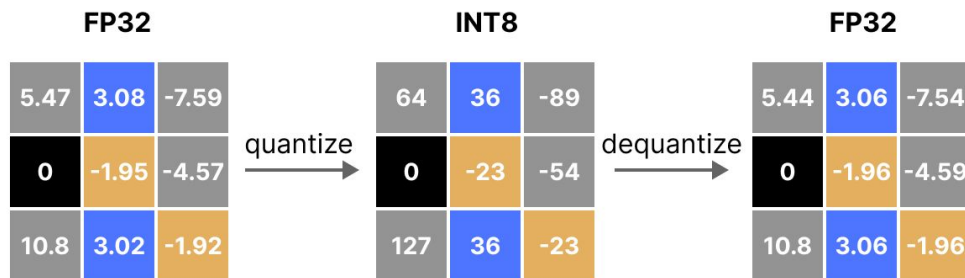
**Всего = (Количество параметров*Точность) + KV
cache + Overhead**



Квантование

Квантование – это

метод сжатия весов и активаций LLM путем уменьшения точности их представления. Позволяет как уменьшить требования модели к размеру памяти, так и увеличить скорость выполнения. Побочным эффектом является понижение точности предсказаний модели.

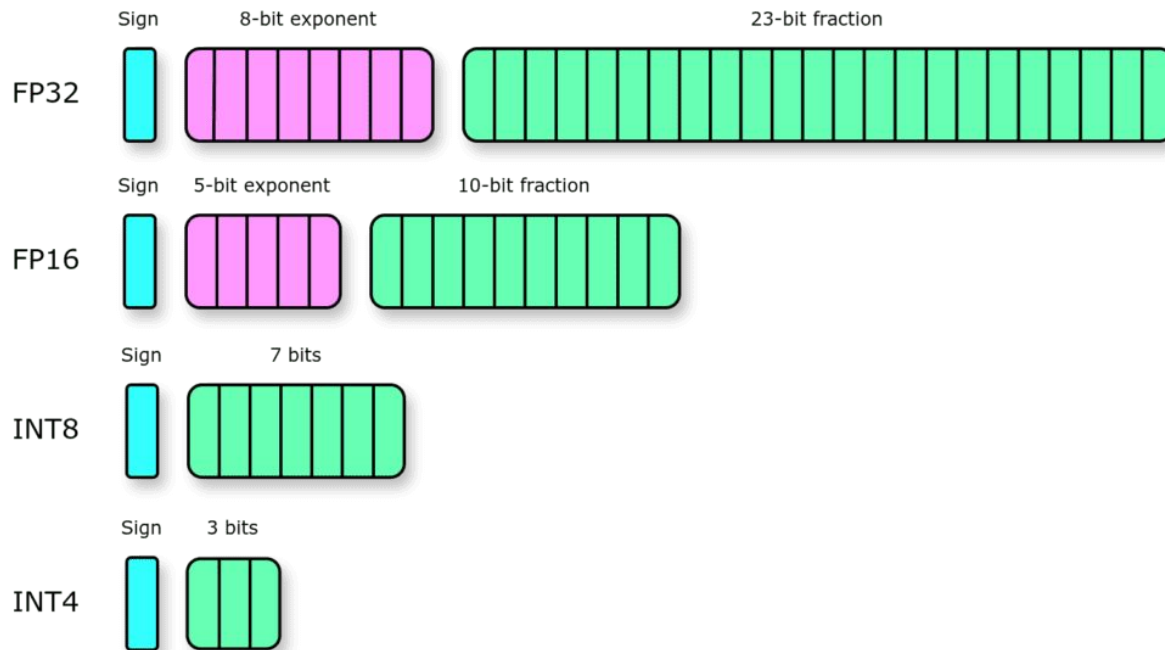


Плюсы и минусы

- значительно снижает требования к памяти, в т.ч VRAM
- за счет меньшего размера весов, смещений и активаций значительно ускоряются memory bounded операции, что в итоге приводит к ускорению инференса

- снижение качества предсказаний моделей, особенно сильное при агрессивных подходах к квантизации
- меньше гибкости в части используемых для инференса инструментов, потенциальные проблемы с совместимостью и стабильностью

Разрядность чисел при квантизации



Различные подходы к квантованию

- Post-Training Quantization (PTQ) – квантование после обучения уже готовых весов
- Quantization Aware Training (QAT) – квантование в процессе обучения модели
- Динамическое квантование – процесс калибровки значений активации в процессе инференса модели, на основании реально вычисленных значений
- Статическое квантование – процесс калибровки (вычисление нулевой точки и коэффициента масштабирования) выполняется заранее, при прогоне на тестовых данных

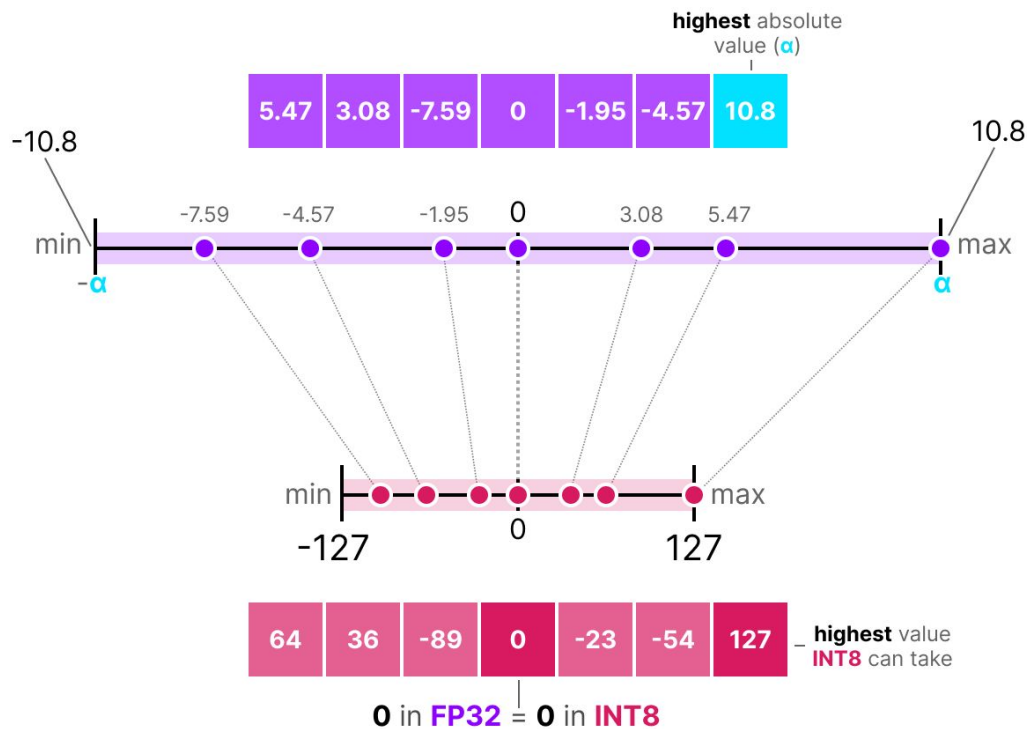
dynamic values
("activations")

$$\begin{array}{c} \text{output} \\ \text{Y} \end{array} = \begin{array}{c} \text{w} \end{array} \begin{array}{c} \text{input} \\ \text{X} \end{array} + \begin{array}{c} \text{b} \end{array}$$

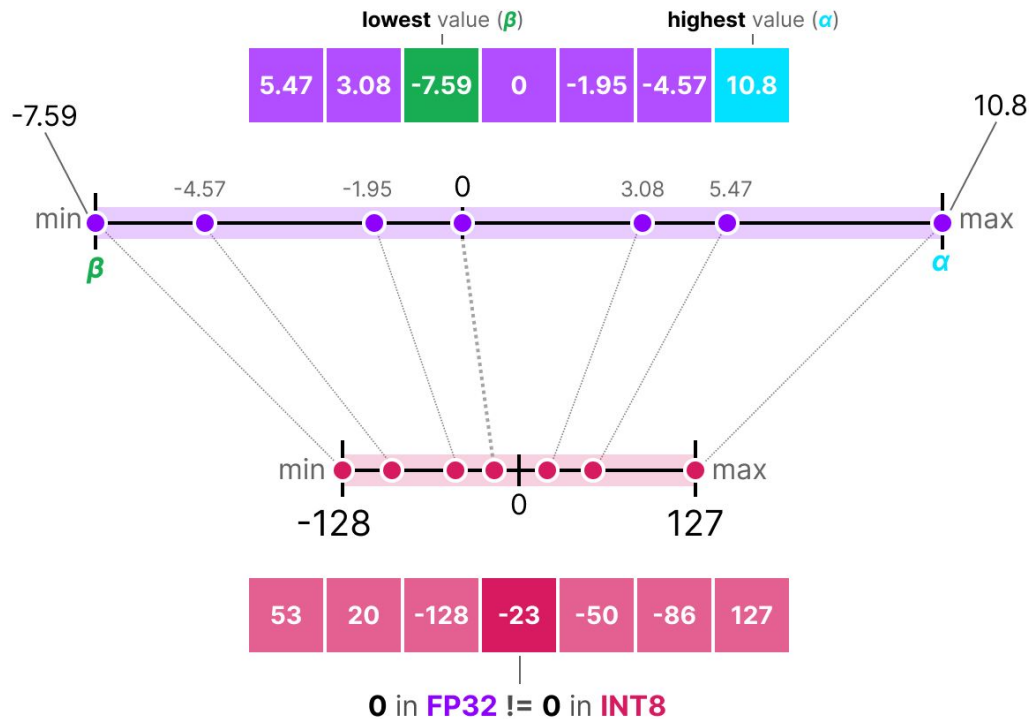
Экстремальные подходы (4 бита и меньше)

- **AWQ** (Activation Aware Quantization) – выявление «сверхвесов» при прогоне на тестовых данных, которые оказывают наибольшее влияние, и предотвращение их чрезмерной квантизации
- **GPTQ** (Generalized Post-Training Quantization) – послойная ассиметричная квантизация, минимизирующая ошибку квантования каждого слоя при помощи градиентных операций и вторых производных функции потерь (обратная матрица Гессiana)
- **GGUF** – метод квантования и формат хранения весов, позволяющий запускать модели на низко производительном оборудовании (загрузка только части весов в VRAM). Используется блочное квантование, для каждого блока подбирается свой набор коэффициентов масштабирования

Симметричное квантование

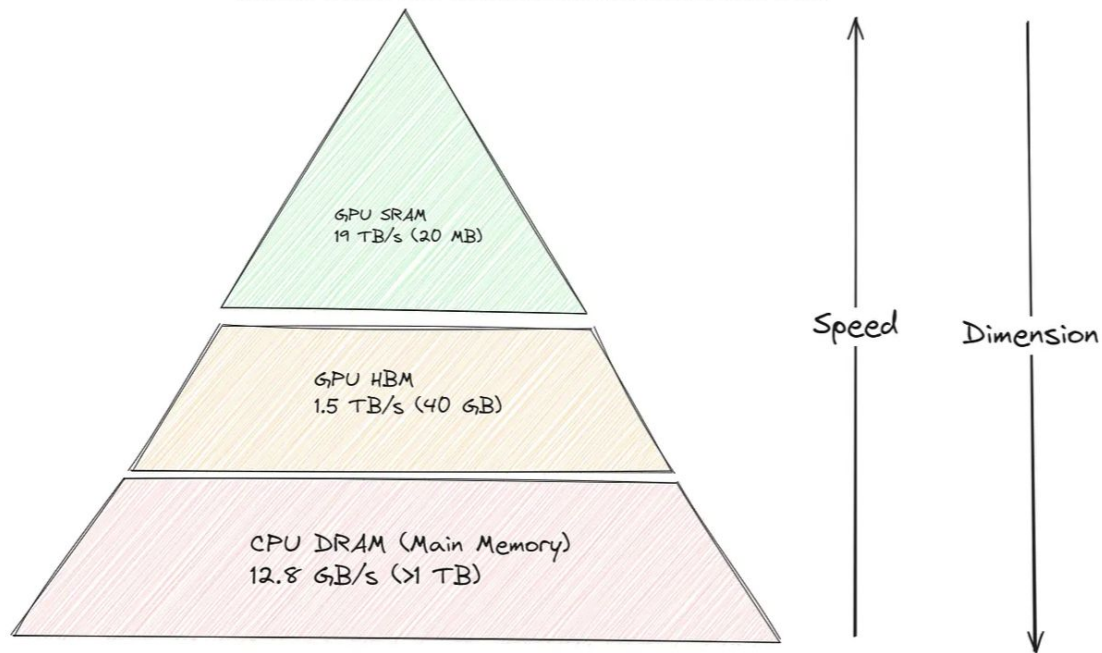


Асимметричное квантование



Техники оптимизации инференса

Структура памяти



That's a lot of wasted time and energy !!!

Flash Attention

- Стандартный подход к вычислению внимания имеет квадратичную сложность и зависит от количества входных данных
- Алгоритм, ускоряющий вычисления механизма внимания как для инференса, так и для обучения моделей
- Разбивает матрицы внимания на блоки, для возможности их обработки в быстрой SRAM
- Минимизирует обращение к медленной памяти
- Необходима поддержка на аппаратном уровне (A100 +)
- Существует несколько версий алгоритма (v1, v2, v3) с разными степенями оптимизации



Особенности работы KV кэша

- Модели генерируют токены итеративно, пока не достигнут END токена или не превысят допустимую длину последовательности
- Для генерации каждого нового токена необходимо выполнить весь набор вычислений
- Для оптимизации операций вычислений можно хранить значения K и V матриц, рассчитанных для предыдущего набора токенов (Q требуется пересчитывать все равно)
- При большом размере контекста размер KV кэша достигает существенных объемов (десятки и сотни гигабайт) и резко возрастают накладные расходы по перемещению значений между разными сегментами памяти
- Память выделяется заранее под максимальных размер контекста и сильно фрагментируется в процессе вычислений



Paged Attention

- Разбиение KV кэша на блоки (страницы)
- Аллокация памяти поблочно по мере необходимости
- Позволяет избегать фрагментации памяти и излишнего ее выделения сразу под максимальный размер контекста
- Стандартный подход работы с KV cache в высокопроизводительных движках инференса (например vLLM)

Запрос с max_tokens=2048:

Выделено: [] 2048 слотов

Реально использовано: [] 200 токенов

Потери: 90% VRAM на пустое место!



Батчинг

- Инференс LLM во многом Memory bound операция, вычислительные мощности ядра GPU редко утилизируются должным образом
- Большая часть времени затрачивается на загрузку весов модели
- Батчинг – подход уровня движка инференса, объединяющий входящие запросы в батчи, для более эффективной утилизации вычислительных ресурсов GPU за счет параллельной обработки нескольких запросов, без лишних операций загрузки/выгрузки из памяти



Статичный (наивный) батчинг

- Позволяет достичь большей утилизации GPU чем без использование батчинга
- Большая часть времени затрачивается на загрузку весов модели
- Сильно зависит от равномерности длин последовательностей в входном батче

T_1	T_2	T_3	T_4	T_5	T_6	T_7	T_8
S_1	S_1	S_1	S_1				
S_2	S_2	S_2					
S_3	S_3	S_3	S_3				
S_4	S_4	S_4	S_4	S_4			

T_1	T_2	T_3	T_4	T_5	T_6	T_7	T_8
S_1	S_1	S_1	S_1	S_1	END		
S_2	S_2	S_2	S_2	S_2	S_2	S_2	END
S_3	S_3	S_3	S_3	END			
S_4	S_4	S_4	S_4	S_4	S_4	END	

Динамический батчинг

- Позволяет достичь большей утилизации GPU чем без использование батчинга
- Не дожидается окончания обработки каждого батча, а выполняет планирование после каждого полученного токена
- Позволяет выполнять prefill и decode фазу для разных запросов независимо

T_1	T_2	T_3	T_4	T_5	T_6	T_7	T_8
S_1	S_1	S_1	S_1				
S_2	S_2	S_2					
S_3	S_3	S_3	S_3				
S_4	S_4	S_4	S_4	S_4			

T_1	T_2	T_3	T_4	T_5	T_6	T_7	T_8
S_1	S_1	S_1	S_1	S_1	END	S_6	S_6
S_2	S_2	S_2	S_2	S_2	S_2	S_2	END
S_3	S_3	S_3	S_3	END	S_5	S_5	S_5
S_4	S_4	S_4	S_4	S_4	S_4	END	S_7

Запуск нескольких моделей на одном GPU

- Не всегда запускаемые модели настолько требовательны к ресурсам, чтобы эффективно использовать всю видеокарту
- MIG профили – способ деления некоторых карт от Nvidia, при котором выделяется изолированный фиксированный объем памяти и вычислительных ядер GPU
- Поддерживается только некоторыми GPU (A100 / H100 / B100 и другие)
- Одну GPU можно разделить вплоть до 7 независимых MIG профилей
- Time slicing – альтернатива для карт, не поддерживающих MIG
- Разделяет ресурсы одного GPU между несколькими задачами, чередуя их выполнение по времени
- Дает некоторый оверхэд на потребление ресурсов и не обеспечивает полной изоляции



Вопросы



если есть вопросы



если вопросов нет

Теперь вы сможете

1. Рассчитывать сайзинг необходимый
для запуска LLM

2. Оптимизировать обучение LLM
моделей

3. Оптимизировать инференс LLM
моделей

